

课程笔记

UCB CS294/194-196: Agentic AI (Fall 2025) - Lecture 02: LLM Agents

Overview

Yann Dubois & Codex

2025-09-15



视频作者/频道: Agentic AI MOOC / UC Berkeley

发布日期: 2025-09-15

视频时长: 1:58:21

视频链接: <https://www.youtube.com/watch?v=r1qZpYAmqmg>

目录

1 LLM 训练流水线与 agents 问题设定	2
1.1 本章小结	2
2 Pretraining: 数据、规模与 scaling laws	3
2.1 拓展阅读	3
2.2 本章小结	4
3 Post-training: SFT、RLHF、RL 与 reasoning RL	4
3.1 本章小结	5
3.2 拓展阅读	5
4 Fall 2024 前传: 从 reasoning 到 environment interaction	5
4.1 本章小结	5
5 Evaluation: 从 perplexity 到 ChatBot Arena	5
5.1 本章小结	7
6 Systems: GPU、精度、fusion、tiling 与 FlashAttention	7
6.1 本章小结	8
7 Tokenizer: BPE 与输入边界	8
7.1 本章小结	8
8 总结与延伸	9
8.1 本章小结	9

1 LLM 训练流水线与 agents 问题设定

这一讲名义上叫 *LLM Agents Overview*，但它实际上是在回答一个更基础的问题：**如何训练一个能做 agent 的 LLM**。Yann Dubois 在开头直接把训练流水线拆成三段：pretraining、classic post-training / alignment，以及更近年的 reasoning reinforcement learning。这个拆法很重要，因为它把“模型怎么变聪明”与“模型怎么变成助手”分开了。

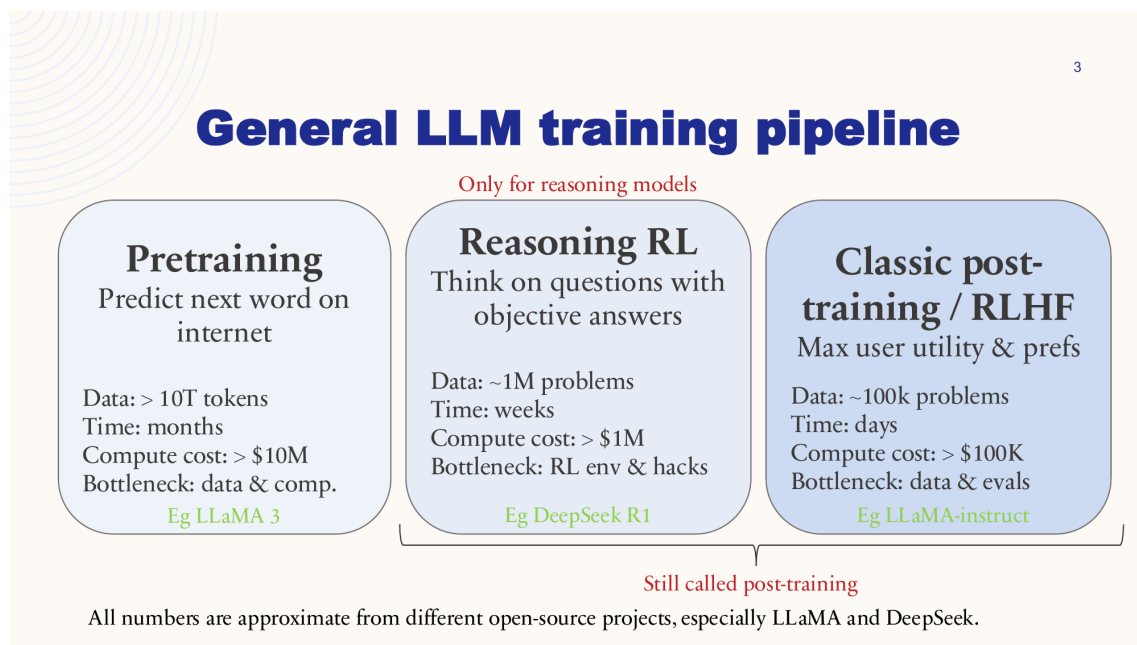


图 1: LLM 训练流水线的三段式视角：pretraining、classic post-training / RLHF，以及 reasoning RL。

pretraining 的对象是互联网文本，目标是 next-word prediction；classic post-training 的对象是人类偏好和指令，目标是让模型更会回答用户；reasoning RL 则把目标进一步推向 hard tasks with verifiable answers，让模型学会在测试时花更多算力做推理。这个顺序不是历史随便拼出来的，而是能力需求从“懂世界”到“会配合人”再到“会在难题上多想一会儿”的自然升级。

三阶段各自回答什么问题

- **Pretraining**: 模型是否学到足够广的世界知识与语言分布？
- **Post-training / alignment**: 模型是否会按人的意图、格式和约束来回答？
- **Reasoning RL**: 当答案可验证时，模型是否愿意在推理阶段花更多计算换正确率？

从 lecture 一开始就可以看出，课程不是把 LLM 视为“纯语言系统”，而是把它视为将来要进入外部环境的决策核心。于是训练的焦点不再只是语言质量，而是 *assistant usefulness*、*tool use*、*evaluation* 和 *systems*。

1.1 本章小结

这一节的主旨是：Agent 不是 pretraining 的副产品，而是 post-training、reasoning 和系统化工具链共同塑造出来的行为形态。理解三段式训练流水线，后面的所有讨论才会有坐标系。

2 Pretraining: 数据、规模与 scaling laws

pretraining 仍然是基础中的基础：模型在这一阶段学习 “everything in the world”，任务是 predict the next word。对大模型来说，训练数据通常超过 10T tokens，时间尺度是 months，算力成本是 millions of dollars。讲者强调，这一阶段的两个瓶颈是 data 和 compute，而且二者都不是简单地“多喂一点”就能解决。

slide 12 到 15 的重点是数据工程。真实的互联网并不是 clean internet，因此预训练数据要经历下载、HTML 抽取、去除 NSFW / harmful content / PII、去重、heuristic filtering、model-based filtering 和 data mix reweighting。lecture 的核心判断很明确：**高质量数据不是附加项，而是预训练里最重要的工程工作之一。**

与此同时，课件用 scaling laws 把“规模化不是盲目堆卡”讲得非常具体。slide 17 说，模型性能可以随着 compute 预测；slide 18 进一步说，训练 recipe 可以先在小模型上调，再外推到大模型。slide 20 的 Chinchilla 结论则给出更具体的资源配比思想：在固定 FLOP 下，数据量和参数量需要平衡，而不是一味追大参数。

为什么 scaling laws 很重要

scaling laws 的意义不是“更大一定更好”，而是：

- 用小规模实验预测大规模结果，减少盲试成本；
- 把超参搜索从大模型迁移到小模型；
- 让训练系统与数据配比成为可优化对象。

slide 21 的 “Bitter lesson” 在这一讲里非常关键。它不是一句口号，而是把整个课程的工程哲学定下来：长期看，真正有效的是 leveraging computation 的通用方法，而不是手工复杂化。slide 22 则给出一个现成的大模型算力账本，例如 LLaMA 3 400B 级别模型训练会进入 10^{25} FLOPs 的量级。

$$\text{FLOPs} \approx 6NP$$

其中：

- N 是训练 token 数；
- P 是参数规模；
- 前面的系数 6 是训练 FLOP 的粗略经验估计。

这一部分的教学意义在于：agent 训练不是抽象算法游戏，而是一个有明确成本上限和数据稀缺性的工程问题。你如果理解不了 pretraining 的数据与算力约束，就很难理解后面为什么 post-training、evaluation 和 systems 会变成课程重点。

2.1 拓展阅读

- *FineWeb*: 高质量 web 数据构建思路。
- *Chinchilla*: tokens-parameters 资源配比的代表性结论。

2.2 本章小结

Pretraining 的核心是把世界知识压进参数里，但它的上限由数据质量和算力共同决定。Scaling laws 的价值在于把训练从“玄学试错”改造成“可外推的实验设计”。

3 Post-training: SFT、RLHF、RL 与 reasoning RL

pretraining 学的是世界，post-training 学的是“怎么像一个助手”。slide 25 说得非常直接：language modeling $\neq$ assisting users。一个预训练得很强的模型，可能会继续做 next-token prediction，却未必知道如何遵守指令、遵守格式、进行工具调用，或者在需要时展示思考过程。

经典 post-training / alignment 的目标是 maximize answer preferences of humans。slide 31 把 supervised finetuning (SFT) 讲得很明白：先收集人类给出的高质量输入输出对，再把它看成语言建模问题。OpenAssistant 被当作这一阶段的代表性例子。SFT 的作用不是让模型“获得全部知识”，而是把它从一般语言模型变成更像助手的模型。

但 slide 32 到 36 很快提醒我们：人类数据慢、贵，而且很容易不够覆盖实际需求。于是出现了 synthetic data generation。Alpaca 用 LLM 扩展数据规模，DeepSeek-R1 用 rejection sampling 和 verifiers 构造更偏 reasoning 的数据，Kimi K2 甚至把 LLM simulated user、tools、rubric-based rejection sampling 组合成复杂的数据流水线。

SFT 学什么

SFT 不只是学“回答问题”：

- instruction following;
- desired format / style;
- tool use;
- early reasoning;
- 任何能构成高质量输入输出对的行为。

slide 38 到 47 进入 RL 与 RLHF。讲者的逻辑是：SFT 本质上是 behavior cloning，但它会被 human limitations、hallucination 和 data price 限制住。RL 则不克隆行为，而是直接最大化 desired behavior。reward 可以来自规则、reward model、LLM-as-a-judge，或者 human preference。RLHF 的经典 pipeline 是：生成候选答案、让标注员或偏好模型选优、再用 PPO 或 DPO 提升偏好得分。

slide 46 和 47 把一个现实转折点讲出来了：人类偏好数据太慢太贵，于是 LLM preferences 成为一种可行替代。它不只是节省成本，也使得很多任务可以进入自动化数据回路。这个判断在 agent 场景尤其重要，因为 agent 数据往往比普通对话数据更昂贵、更依赖环境。

RL 不是只为了“更像人”

在这门课里，RL 更大的作用是把模型从模仿者推向优化器：当输出质量可以被规则、测试或 judge 约束时，模型就有机会直接优化目标，而不是只复刻人类平均行为。

3.1 本章小结

post-training 这部分的关键，不是某一个具体方法，而是从 SFT 到 RLHF 再到 reasoning RL 的目标迁移：从“像人”走向“对任务有用”，再走向“在可验证任务上真正想办法”。这正是 agent 能力的训练底座。

3.2 拓展阅读

- *KIMI K2: Open Agentic Intelligence*
- *DeepSeek-V3 Technical Report*

4 Fall 2024 前传：从 reasoning 到 environment interaction

如果把这门课的 Fall 2025 讲义单独看，读者很容易把 agent 理解成“LLM + 工具调用”的后缀。但 Berkeley Fall 2024 的前导课程补了一层更基础的认知：一个真正能做 agent 的模型，首先要会在多步推理中保持稳定，然后才谈得上与环境交互。

‘LLM Reasoning’ 讲的是 serial reasoning 的边界：chain-of-thought、self-correction、premise order sensitivity，以及 reasoning 何时需要 verifiers 或更强的 scaffolding。它给这门课提供了一个底层前提：如果模型在推理层本来就不稳，那么 agent 的外层 workflow 只能是补丁，不可能是稳固系统。

‘LLM agents: brief history and overview’ 则把问题推进到 WebShop 和 ReAct。前者告诉我们，agent 的成功标准不是句子好不好，而是任务有没有完成；后者把 reasoning trace 和 action trace 合并，让“想”和“做”成为同一条轨迹。换句话说，Fall 2024 明确地把 agent 定义成 *environment-interacting system*，而不是更长的 chatbot prompt。

Fall 2024 对 Fall 2025 的前导作用

- 先理解 reasoning 的稳定性与局限，再理解 agent 的执行闭环；
- 先理解 task completion 与 environment feedback，再理解 evaluation 与 systems；
- 先把 agent 看成可执行系统，再把它看成高阶助手。

4.1 本章小结

Fall 2024 的补前导不是在重复同一知识，而是在帮读者把“reasoning → interaction → workflow”这条链路接起来。这样看后面的 post-training、evaluation 和 systems，agent 的定义会更完整。

5 Evaluation：从 perplexity 到 ChatBot Arena

lecture 里对 evaluation 的强调非常强。原因很简单：如果你不能稳定测量模型能力，就无法稳定改进它。slide 49 直接说 evaluation 的作用是 quantify progress，进而 identify improvements、select models、decide production readiness。

closed-ended evaluation 的优点是可自动验证。MMLU 是代表性的例子：答案集合有限，系统可以自动算分。perplexity 也是类似的 development metric，它衡量的是模型对下一个 token 的平均不确定性。一个更标准的写法是：

$$\text{PPL}(x_{1:L}) = \exp\left(\frac{1}{L} \sum_{i=1}^L -\log p(x_i | x_{<i})\right)$$

其中：

- $x_{1:L}$ 是长度为 L 的 token 序列；
- $p(x_i | x_{<i})$ 是模型在前文条件下对第 i 个 token 的概率；
- PPL 越小，表示模型对下一个 token 越不“犹豫”。

slide 75 到 77 进一步强调：perplexity 曾经非常重要，但它已经不是最终学术 benchmark 的中心，因为它容易和真正的 assistant utility 脱节；而评测还有 prompt sensitivity、train-test contamination、spurious correlation 等问题。比如 AlpacaEval 会偏好更长的输出，所以需要 regression / causal inference 去控制长度偏差。

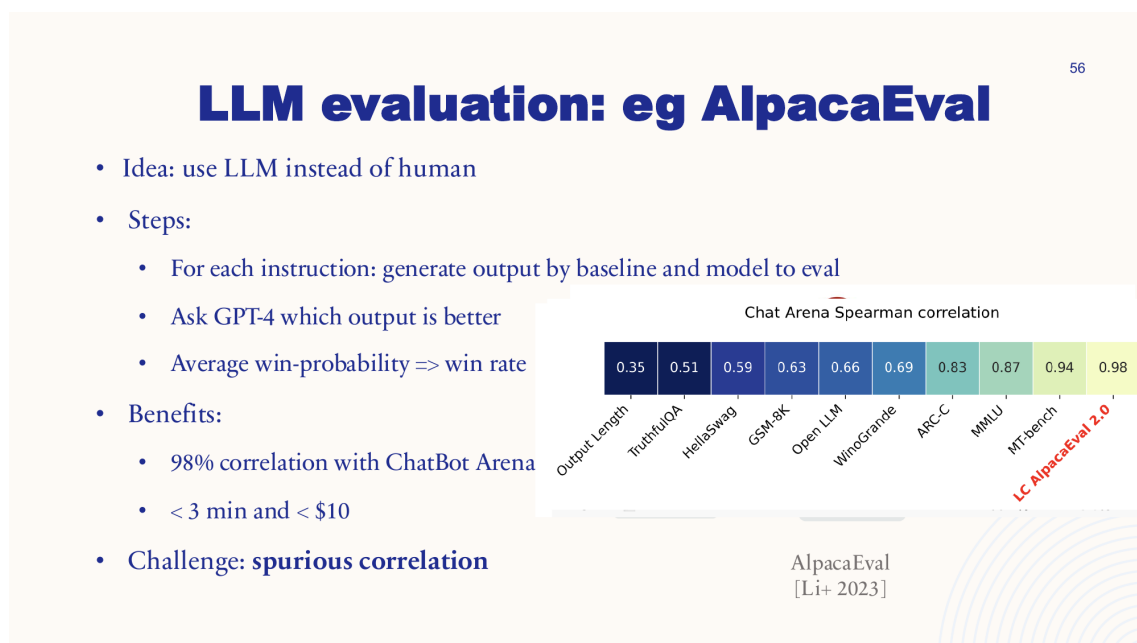


图 2: LLM evaluation 的代表性路径：用 LLM 代替人类做 pairwise judge 的 AlpacaEval。

open-ended evaluation 更接近真正的 chat assistant。ChatBot Arena 让用户 blind compare 两个机器人，靠人类偏好来形成 ranking；AlpacaEval 则用 GPT-4 做 judge，试图在成本和相关性之间找到折中。这个部分的核心不是“谁比谁更先进”，而是：**agent 时代的评测必须兼顾自动化、真实性和可复现性。**

为什么 eval 是 agent 的瓶颈

- closed-ended 任务容易自动化，但覆盖不了复杂交互；
- open-ended 任务更接近真实用户，却很难标准化；
- 评测若不稳定，训练目标就会漂移，系统也很难工程化。

5.1 本章小结

Evaluation 不是训练之后的附属工作，而是整个 agent 系统的目标定义方式。Perplexity、ChatBot Arena、AlpacaEval 分别代表了从语言建模到人类偏好再到开放式助手评价的不同层次。

6 Systems: GPU、精度、fusion、tiling 与 FlashAttention

从 slide 57 开始，讲义切换到 systems。这里讲者的基本判断是：大家都被 compute 卡住了，因此 agent 时代真正的瓶颈越来越像系统问题，而不是“再堆几个参数”。GPU 的优势在于 massively parallel 和 fast matrix multiplication, 但劣势也同样明显：compute 高于 memory 和 communication, 很多时候真正卡的是数据搬运。

GPU 相关的第一个核心概念是 *memory hierarchy*：离计算单元越近的存储越快，但容量越小；越远越大，但越慢。MFU (Model Flop Utilization) 则是衡量实际吞吐和理论峰值的比值。简单说，50% 的 MFU 在这门课里就已经被认为是不错的结果了。

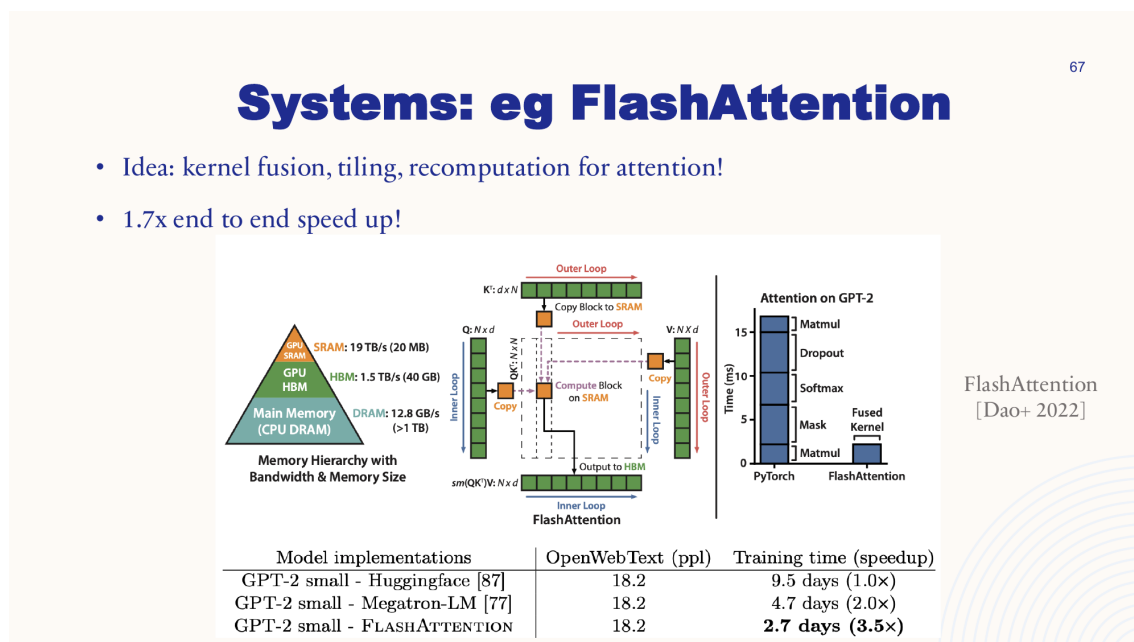


图 3: FlashAttention 代表了“kernel fusion + tiling + recomputation”的系统思路。

低精度训练是第二个关键点。讲者强调，深度学习里 decimal precision 并不是最核心的东西，除了 exp 和 updates 之外，大多数矩阵乘法都可以用 bf16 来做。AMP 的典型逻辑是：权重保留 fp32，计算时转 bf16，激活和梯度在低精度中处理，最后 master weights 用 fp32 更新。这样既省内存，又提高速度。

operator fusion 与 tiling 是系统优化的两个基础动作。前者的目标是减少把中间变量反复写回 global memory；后者的目标是把计算切成适合 cache / SRAM 的小块，在块内尽量重用数据。FlashAttention 就是这两个思路的代表：attention 可以通过 kernel fusion、tiling 和 recomputation 显著提速，而不必把所有中间矩阵都落回慢速内存。

两条系统规律

- **少搬数据**：能在片上算，就不要频繁写回 global memory。
- **多复用数据**：把算子切成 tile，先把局部块算满，再换下一块。

最后是 parallelism。slide 68 到 72 说明了为什么大模型训练不能只靠一张卡。naive training 一个 P 参数模型，通常需要约 $16P$ GB DRAM：模型权重、Adam optimizer states 和 gradients 共同吞掉内存。data parallelism 的思路是复制模型、切分数据、最后同步梯度；它的优点是简单，但不带来 memory gains。ZeRO 则通过 sharding optimizer states / gradients / parameters 来减少单卡内存占用。model parallelism 则把层或矩阵切到不同 GPU 上，以支持更大的 batch 或更大的模型。

$$M_{\text{naive}} \approx 16P \text{ GB}$$

其中：

- P 表示参数规模（以 billions 量级理解最直观）；
- $4P$ 是模型权重；
- $8P$ 是 Adam 的均值和方差状态；
- $4P$ 是 gradients。

6.1 本章小结

systems 这一节告诉我们：agent 能不能跑起来，往往取决于 GPU、内存层次、算子融合、并行策略与精度设计，而不是只取决于模型是否更“聪明”。这也是为什么 agent 课程必须讲 infra。

7 Tokenizer：BPE 与输入边界

lecture 的最后几页重新回到 tokenizer。它看起来像基础知识，但实际上它决定了模型真正处理的单位是什么。BPE (Byte Pair Encoding) 把 token 视为常见子序列：先把大语料切成字符级 token，再反复合并最常见的 token pair，直到达到目标 vocab size。

tokenizer 的两个动机是：

- 它比 word 更 general，可以处理 typo 和未登录词；
- 它比 character 更短，能显著缩短序列长度。

lecture 强调的不是实现细节，而是一个更大的结论：**tokenization 是模型输入边界的定义**。后面所有关于 context length、pretraining compute、evaluation 和 agent rollout 的讨论，最后都要落回 token 这个计量单位。

7.1 本章小结

Tokenizer 是 agent 训练链条的最前端约束。BPE 并不神秘，但它决定了序列长度、上下文成本和许多系统优化的起点。

8 总结与延伸

这一讲的最大价值，是把 agent 课程的核心地图串了起来：

1. 训练流水线从 pretraining 走到 post-training，再走到 reasoning RL；
2. evaluation 决定什么叫 progress；
3. systems 决定 progress 能不能以可接受成本落地；
4. tokenizer 这种“看似基础”的东西，实际决定了整个 pipeline 的边界条件。

如果把这一讲压缩成一句话，那就是：*agent* 不只是模型的行为模式，而是训练、评测和系统共同塑造出来的可执行能力。

8.1 本章小结

Lecture 02 把 agent 课程从“概念导论”推进到“训练-评测-系统的一体化框架”。理解这一讲，后面的 tool use、planning、memory、coding agents 和 infrastructure 才能真正连成一条线。